Input and Output

Chapter 3

Chapter Topics

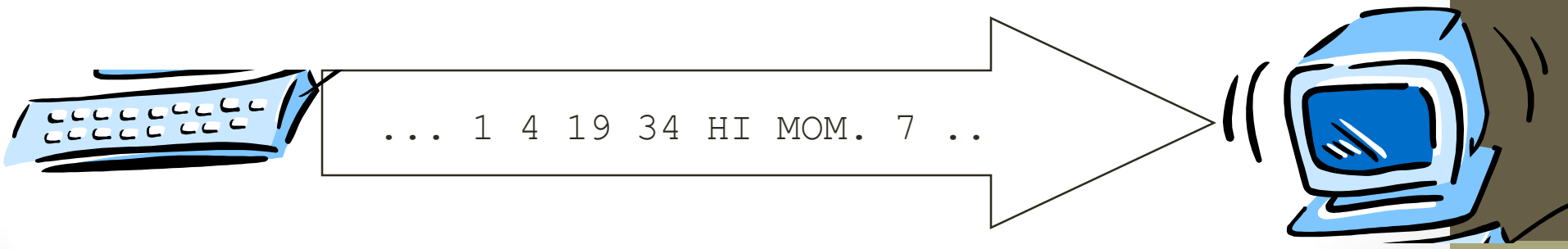
- I/O Streams, Devices
- Predefined Functions
- Input Failure
- Formatting Output
- Formatting Tools
- File I/O

I/O Streams, Devices

- Input stream
  - A sequence of characters/bytes
  - From an input device
  - Into the computer
- Output stream
  - A sequence of characters/bytes
  - From the computer
  - To an output device

Input Streams

- Input data is considered to be an endless sequence of characters/bytes coming into a program from an input device (keyboard, file, etc.)



`cin` and the Extraction Operator >>

- A binary operator
  - Takes two operands
  - Name of an input stream on the left
    - `cin` for standard input from keyboard
  - Variable on the right
- Variables can be "cascaded"

```
cin >> amount >> count >> direction;
```

- Variables should generally be simple types

The Extraction Operator >>

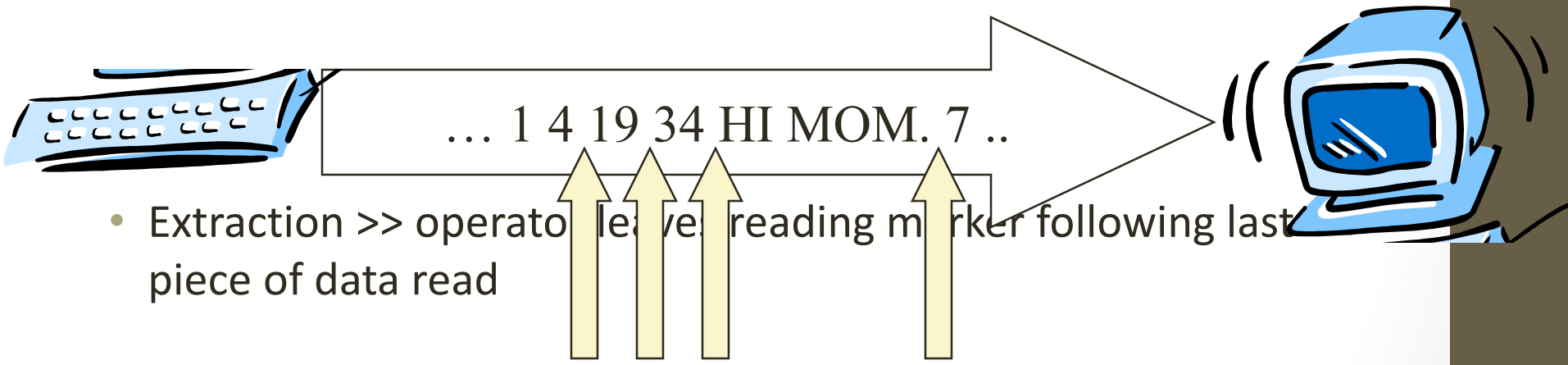
- Enables you to do input with the cin command
- Think of the >> as pointing to where the data will end up
- C++ able to handle different types of data and multiple inputs correctly

```
int main()
{
  float radius, height ;
  cout << "For cylinder volume, enter radius -> ";
  cin >> radius;
  cout << "Enter height -> ";
  cin >> height;
  cout << "Volume = "<<
     3.14 * radius * radius * height;
  return 0;
}
```

```
  float radius, height ;
  cout << "For cylinder volume, enter radius, height -> ";
  cin >> radius >> height;
  cout << "Volume = "<<
     3.14 * radius * radius * height;
  return 0;
}
```

The Reading Marker

- Keeps track of point in the input stream where the computer should continue reading



- Extraction >> operator leaves reading marker following last piece of data read

The Reading Marker

- During execution of a cin command
  - as long as it keeps finding data, it keeps reading
  - when the reading marker hits something **not** data, it **quits** reading
- Things in the input stream that cin considers not data
  - spaces
  - tab \t
  - newline character \n
    (pressing the RETURN key)
  - for numeric input, something **non**numeric

Input to a Simple Variable

- char
  - Skips any white space characters
  - Reads one character
  - Any other characters in the stream are held for later input
- int
  - Skips white space characters
  - Reads leading + or -
  - Reads numerals
  - Quits reading when it hits non numeral

Input to a Simple Variable

- double (or float)
  - Skips leading white space
  - Reads leading + or –
  - Reads numerals and at most one decimal point
  - Quits reading when it hits something not numeric
- Note example, page 95

Predefined Functions

- Function in a computer language is <u>similar</u> to concept of a function in mathematics
  - The function is sent value(s)
    - Called "arguments" or "parameters"
  - It manipulates the value
  - It returns a value
- Some functions "do a task"

Reading `cString` Data with `cin`

- Keyboard response of **two** words (separated by a space) causes the cin command to quit reading
  - the space is considered **non**data (in spite of our intent)

```
#include <iostream.h>

int main()
{
   char student_name[30];
   cout << "Enter name -> ";
   cin >> student_name;
   cout << "Hello, " << student_name;
   return 0;
```

```
(Inactive C:\CLASS...
Enter name -> Snidly Fizbane
Hello, Snidly
```

???

Reading `cString` Data

- The `getline ( )` function will allow the programmer to access all the characters

```
int main()
{
   char student_name[30];
   cout << "Enter name -> ";
   cin.getline (student_name,30,'\n');
   cout << "Hello, " << student_name;
   return 0;
```

```
(Inactive C:\CLASSES\COSC13...
Enter name -> Mortimer Q. Snarfblat
Hello, Mortimer Q. Snarfblat
```

Character which terminates read

`char` Array variable

Length (max number of characters)

`cin` and the `get` Function

- Syntax:
  `cin.get(varChar);`
- Example
  `cin.get (chVal);`
  - `chVal` is the parameter
  - the .`get` function retrieves a character from the keyboard
  - stores the character in `chVal`

`cin` and the `ignore` Function

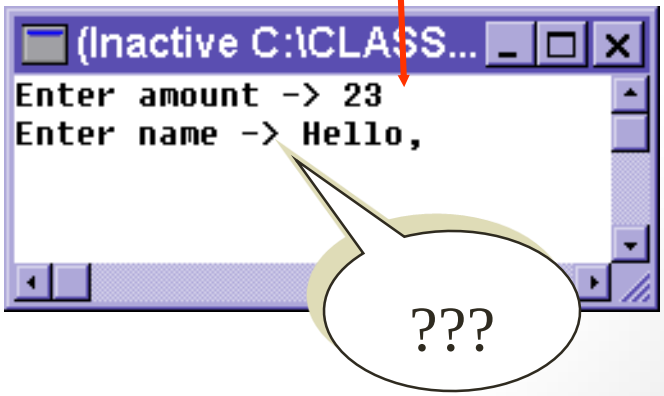
- Syntax:
  `cin.ignore (intValue, charVal);`
- Example:
  `cin.ignore (10,'\n')`
- The ignore function causes characters in the input stream to be `ignore`d (discarded)
  - In this example for 10 characters … or …
  - Until a newline character occurs
  - It also discards the newline character

The **`ignore`** and **`get`** also work for other input streams (such as file input streams)

Using the getline( )

- Problem : the getline( ) quits reading when it finds a newline
  - Suppose you have terminated previous input with the <RETURN> key (newline still in input stream)
  - getline ( ) finds the newline **immediately** and declares its task finished
  - we must somehow discard the newline in the input stream

```
int main()
{ int amount;
  char student_name[30];
  cout << "Enter amount -> ";
  cin >> amount;
  cout << "Enter name -> ";
  cin.getline (student_name,30,'\n');
  cout << "Hello, " << student_name;
  return 0;
}
```



Using the ignore( )

- Solution : the ignore( ) command
- Tells the program to skip either the next 10 characters or until it reaches a newline
  - whichever comes first
- This effectively discards the newline

```
int main()
{ int amount;
  char student_name[30];
  cout << "Enter amount -> ";
  cin >> amount;
  cin.ignore (10,'\n');
  cout << "Enter name -> ";
  cin.getline (student_name,30,'\n');
  cout << "Hello, " << student_name;
  return 0;
```

```
(Inactive C:\CLASSES\CO...
Enter amount -> 187
Enter name -> I. M. Bicpentamiter
Hello, I. M. Bicpentamiter
```

Input Failure

- Happens when value in the input stream is invalid for the variable

```
int x, y;
cin >> x >> y;   //  Enter  B  37
```

- Value of 'B' not valid for an int

- View example

When an input stream fails system ignores all further I/O

The `clear` Function

- Use the clear to return the input stream to a working state
- Example
  look for
  ```
  cin.clear()
  cin.ignore (200,'\n');
  // to empty out input stream
  ```

Formatting Output

- Producing proper output in the proper format is important
  - Specify decimal precision
  - Specify left or right justification
  - Align columns of numbers
- C++ provides I/O manipulators
- Syntax:

```
cout << manipulator << expression
…
```

Manipulators

- Must first of all
  `#include <iomanip>`
- For decimal precision use
  `cout << setprecision (n) << …`
- To output floating point numbers in fixed decimal format use
  `cout << fixed << …`
- To force decimal zeros to show
  `cout << showpoint << …`

Manipulators

- To specify right justification in a specified number of blanks use
  `cout << setw(n) << …`
- If the number of blanks required to print the expression exceeds specified size
  - Size is ignored
- Problem – print series of names left justified followed by right jus
  Os
  Jo

•Names are of different length

•Need variable number of spaces

Manipulators

- Print name, then variable number of spaces using the setw( )
- Example

```
cout << showpoint << fixed ;
cout << name <<
        setw( 25 - strlen(name))<<"  ";
cout << setw (8) <<
        setprecision(2) << amt;
```

Formatting Tools

- Possible to specify a character to fill leading spaces

```
cout.fill ('*');
cout << setw(10) <<
        setprecision(2);
cout << pmtAmount ;
```

- Result

```
*****12.34
```

File I/O

- Previous discussion has considered input from the keyboard
  - This works fine for limited input
  - Larger amounts of data will require file input
- File:
  An area of secondary storage used to hold information
- Keyboard I/O `#include <iostream>`
- File I/O `#include <fstream>`

File I/O

- Requirements to do file I/O

1. `#include <fstream>`
2. Declare a file stream variable
   ifstream or ofstream
3. Open the file – use the command
   `whateverFile.open("filename.xxx");`
4. Use the stream variable with `>>` or `<<`
5. Close the file `whateverFile.close();`

File Open Modes

- In some situations you must specify one or more modes for the file
- Syntax:

```
fileVariable.open("filename.xxx",
fileOpenMode);
```

File-Opening Mode	Description
`ios::in`	Opens the file for input. This is the default for `ifstream` variables.
`ios::out`	Opens the file for output. This is the default for `ofstream` variables.
`ios::nocreate`	If the file does not exist, the open operation fails.
`ios::app`	If the file exists, adds the output at the end of the file (append). If the file does not exist, creates an empty file.
`ios::ate`	Opens the file for output and moves to the end of the file. Output may be written anywhere in the file.
`ios::trunc`	If the file exists, the contents will be deleted (truncated).
`ios::noreplace`	If the file exists, the open operation fails.

Using Files in Programs

- Specify `#include <fstream>` header file
- Declare instance of file to be used
- Prepare for access with `.open( )` command
- Use name of file in place of cin or cout

```
#include <fstream>

int main()
{
   int a, b;
   ifstream infile;
   infile.open ("values.txt");
    . . .

   infile >> a >> b;
    . . .
   return 0;
```

file name on disk